Gesture Recognition from MediaPipe Hand Landmarks

A deep, self-contained study guide for File 3 of a custom gesture camera application

> Pipeline assumed: OpenCV → MediaPipe Hand Landmarker → File 3 → gesture → camera action

Goal: finish this guide able to design, implement, test, and improve a robust landmark-based gesture recognition system, including custom static gestures and motion gestures.

What this guide covers

- 21-landmark geometry and coordinate systems
- Translation, scale, rotation, handedness, and depth normalization
- Vectors, dot products, cross products, angles, distances, and ratios
- Finger-state detection and geometric rule systems
- Feature engineering and feature normalization
- KNN, SVM, Random Forest, MLP, and model selection
- Static versus dynamic gestures
- Temporal windows, smoothing, majority voting, debounce, cooldowns
- Dynamic Time Warping (DTW) and sequence features
- Dataset design, labeling, train/validation/test splits, augmentation
- Confidence, rejection, false positives, latency, and evaluation
- A clean Python architecture for your File 3
- A practical project roadmap and experiments to run

Prerequisites: basic Python, NumPy, basic algebra, and familiarity with OpenCV/MediaPipe. The mathematical sections explain the required concepts from scratch.

Contents

#	Section	Page
1	The role of File 3	4
2	What the 21 landmarks mean	5
3	Coordinate systems and representations	6
4	Normalization: the core problem	8
5	The mathematics you need	11
6	Feature engineering	15
7	Rule-based gesture recognition	19
8	Machine-learning classifiers	22
9	Static custom gestures	26
10	Dynamic gestures and time	29
11	Temporal stability and trigger logic	33
12	Handedness, depth and difficult cases	36
13	Dataset design and evaluation	39
14	Software architecture for File 3	43
15	Implementation blueprint	47
16	Experiments and learning roadmap	52
17	Debugging checklist	56
18	Further study and references	59

1. The role of File 3

File 1 deals with images/camera frames. File 2 deals with locating the hand and producing landmarks. File 3 should not care about pixels. Its job is to convert landmarks into a representation that is useful for deciding which gesture is being performed.

The key idea is to separate representation from classification. First turn raw landmarks into a stable geometric description. Then classify that description. Finally, a separate controller maps a confirmed gesture to an action such as taking a photo or starting/stopping video.

Do not put camera actions inside the classifier. If the classifier says “peace, 0.94 confidence,” another layer should decide whether that gesture is mapped to “take photo.” This makes custom mappings possible without retraining the recognizer.

• Input: MediaPipe hand landmarks, handedness, and optionally world landmarks.

• Processing: clean → normalize → extract features → classify → smooth over time → return a GestureResult.

• Output: a structured result such as name, confidence, hand, stability, and optionally motion/state information.

• Keep the system deterministic and testable: the same landmark input should produce the same File 3 output.

2. What the 21 landmarks mean

MediaPipe Hands/Hand Landmarker represents a hand with 21 named landmarks. The common index layout is shown below. Memorize the structure rather than memorizing isolated numbers.

Each landmark has image-space x/y values and depth information; MediaPipe can also expose world landmarks. Your code should define constants instead of scattering numbers like landmarks[8] throughout the project.

Landmark	Index	Landmark	Index
Wrist	0	Thumb CMC	1
Thumb MCP	2	Thumb IP	3
Thumb TIP	4	Index MCP	5
Index PIP	6	Index DIP	7
Index TIP	8	Middle MCP	9
Middle PIP	10	Middle DIP	11
Middle TIP	12	Ring MCP	13
Ring PIP	14	Ring DIP	15
Ring TIP	16	Pinky MCP	17
Pinky PIP	18	Pinky DIP	19
Pinky TIP	20	—	—

3. Coordinate systems and representations

Suppose a landmark is $P_i = (x_i, y_i, z_i)$. The most important question is: what should remain the same when the person moves their hand? A gesture should normally remain recognizable when the hand moves around the image, changes size, and rotates.

Image-space coordinates are useful for screen position and interaction with the camera frame. They are less suitable as the sole gesture representation because absolute x/y changes when the hand moves.

Relative coordinates describe landmarks relative to an anchor, commonly the wrist or palm center. They remove translation.

World landmarks provide a 3D representation that can be useful for shape and depth. Do not assume that every application should use world coordinates exclusively; benchmark both representations.

• Translation invariance: same hand shape should work anywhere in the frame.

• Scale invariance: the same hand shape should work near and far from the camera.

• Rotation invariance: the same gesture should ideally survive reasonable wrist/hand rotation.

• Handedness invariance: left and right hands may need to map to a common canonical representation.

• Depth robustness: gestures that differ in 3D may look similar in a 2D projection.

```
# Use named constants, not magic numbers
WRIST = 0
INDEX_MCP, INDEX_PIP, INDEX_DIP, INDEX_TIP = 5, 6, 7, 8
MIDDLE_MCP, MIDDLE_PIP, MIDDLE_DIP, MIDDLE_TIP = 9, 10, 11, 12
```

4. Normalization: the core problem

4.1 Translation normalization. Choose an anchor A, usually the wrist. For each point P_i, calculate $P_i' = P_i - A$. The wrist becomes (0,0,0). A hand moving from one part of the frame to another now has the same relative geometry.

4.2 Scale normalization. Choose a scale S such as the wrist-to-middle-MCP distance or a robust palm-size estimate. Then $P_i'' = P_i'/S$. A small and large hand become comparable.

4.3 Rotation normalization. Estimate a reference direction, for example wrist → middle MCP or a palm axis. Compute its angle θ and rotate all centered points by $-\theta$. In 3D, a full orientation frame can be constructed from multiple non-collinear palm vectors.

4.4 Handedness normalization. If left and right hands should behave identically, mirror one representation into a canonical hand. Be explicit about the camera's mirroring and MediaPipe's handedness assumptions.

4.5 Do not over-normalize. If the user's gesture intentionally depends on direction relative to the camera or body, removing that information can destroy useful signal. Decide which invariances your product actually needs.

```
import numpy as np

def normalize_translation(points):
p = np.asarray(points, dtype=float)
return p - p[0] # wrist as anchor

def normalize_scale(points, eps=1e-8):
p = np.asarray(points, dtype=float)
scale = np.linalg.norm(p[9] - p[0])
return p / max(scale, eps)
```

Property	Problem without normalization	Typical solution
Translation	Same gesture at different image locations	Subtract anchor
Scale	Hand appears different when closer/farther	Divide by characteristic size
Rotation	Same pose at different wrist angles	Canonical rotation/frame
Handedness	Left/right may mirror geometry	Canonical mirroring
Depth	2D projections can hide 3D shape	Use z/world landmarks where useful

5. The mathematics you need

Vectors. A bone can be represented as a vector from one landmark to another: $v = P_2 - P_1$. Vector magnitude is $||v|| = \text{sqrt}(v_x^2 + v_y^2 + v_z^2)$.

Distance. The Euclidean distance between A and B is $||A - B||$. Distances are excellent features for relationships such as thumb-tip to index-tip pinch distance.

Dot product. For vectors a and b, $a \cdot b = a_x b_x + a_y b_y + a_z b_z$. It tells you how aligned they are. The angle satisfies $\cos \theta = (a \cdot b)/(||a||||b||)$.

Cross product. In 3D, $a \times b$ gives a vector perpendicular to both. Its magnitude is $||a \times b|| = ||a||||b|| \sin\theta$. Cross products are useful for constructing palm coordinate frames and describing orientation.

Angles at joints. For joint B with neighbors A and C, use vectors $u = A - B$ and $v = C - B$. Then $\theta = \arccos(\text{clamp}((u \cdot v)/(||u||||v||), -1, 1))$. Always clamp because floating-point noise can produce values slightly outside $[-1,1]$.

Ratios. Raw distances can still encode scale. Ratios such as d(tip,wrist)/d(MCP,wrist) are often more robust than raw pixels.

Numerical stability. Guard divisions by zero, missing landmarks, NaNs, and nearly collinear vectors. A real-time system must fail gracefully rather than crash.

```
def distance(a, b):
return np.linalg.norm(np.asarray(a) - np.asarray(b))

def angle(a, b, c, eps=1e-8):
# angle ABC
u = np.asarray(a) - np.asarray(b)
v = np.asarray(c) - np.asarray(b)
den = np.linalg.norm(u) * np.linalg.norm(v)
if den < eps:
return 0.0
cos_theta = np.dot(u, v) / den
cos_theta = np.clip(cos_theta, -1.0, 1.0)
return np.degrees(np.arccos(cos_theta))
```

6. Feature engineering

Raw normalized landmarks are one feature family. Geometry-derived features are another. A strong recognizer often combines several families rather than trusting a single measurement.

Coordinate features: centered/scaled x, y, z for each landmark. For 21 points this is up to 63 values.

Bone-vector features: vectors along each finger segment and selected palm connections. They describe orientation.

Joint-angle features: PIP/DIP/MCP angles summarize whether fingers are straight or curled.

Distance features: fingertip-to-fingertip, fingertip-to-palm, and selected pairwise distances.

Finger-state features: continuous scores or binary open/closed estimates. Prefer continuous scores internally and threshold only when needed.

Temporal features: velocity, acceleration, direction change, trajectory, and feature differences between frames.

Feature family	Examples	Strength	Risk
Normalized coordinates	x/y/z of all landmarks	Rich geometry	Higher dimensional
Angles	finger joint angles	Rotation-resistant	Can lose global position/orientation
Distances	tip-to-tip, tip-to-palm	Simple and robust	Can be ambiguous
Ratios	distance ratios	Scale robust	Bad denominator can be unstable
Finger states	open/closed scores	Interpretable	Threshold-sensitive
Motion	Δx, Δy, velocity	Dynamic gestures	Requires time/history

7. Rule-based gesture recognition

Rule-based recognition is the best place to start because every decision is explainable. You can build a baseline before introducing machine learning.

For example, a pointing gesture can be described by an extended index finger and relatively curled middle/ring/pinky fingers. A pinch can be described by a small thumb-index distance. An open palm can be described by several extended fingers.

However, avoid rules that depend on one image-axis coordinate such as tip.y < pip.y. Such rules break when the hand rotates. Prefer joint angles, relative distances, and vectors.

Use hysteresis where a threshold can flicker. For example, enter PINCH below 0.25 normalized distance and leave PINCH above 0.32. Two thresholds create a stable band.

• Start with 4–6 obvious gestures and build a confusion matrix.

• Log the measurements behind every decision; do not debug only from the final label.

• Use thresholds derived from recorded data rather than guessing permanent constants.

• Rule systems are excellent for safety/rejection logic even if an ML classifier is later used for recognition.

```
def pinch_score(points, thumb_tip=4, index_tip=8, scale=1.0):
d = distance(points[thumb_tip], points[index_tip]) / max(scale, 1e-8)
# Example only; tune from data rather than copying this threshold.
return d
```

8. Machine-learning classifiers

Once your feature vector is stable, you can treat gesture recognition as a supervised classification problem. Each training example is a feature vector X and a label y.

KNN: stores training examples and classifies a new sample from nearby examples. Very easy to understand and useful for small personalized gesture sets. It can become expensive as the dataset grows.

SVM: finds separating boundaries between classes. Often strong on medium-sized, structured feature sets. Scaling features and choosing a kernel matter.

Random Forest: combines many decision trees. It handles nonlinear relationships and mixed features well and is easy to inspect.

MLP: a small neural network can learn nonlinear boundaries. It can be effective, but it adds training/deployment complexity.

Do not choose a model because it sounds advanced. Establish a simple baseline first, then compare models using the same held-out data.

Model	Good starting use	Advantages	Limitations
KNN	Personalized small gesture set	Simple, no heavy training	Inference grows with data
SVM	Static pose classification	Strong on structured features	Needs tuning/scaling
Random Forest	Mixed engineered features	Nonlinear, interpretable	Can be larger than necessary
MLP	Larger custom gesture sets	Flexible nonlinear model	More tuning/data needed

9. Static custom gestures

For a custom gesture, collect multiple examples rather than saving one prototype. One pose varies because of finger position, camera distance, rotation, lighting, hand anatomy, and landmark noise.

A practical recording session can capture several seconds of the gesture, then sample frames. Save the extracted feature vectors plus metadata such as label, handedness, and recording/session ID.

Keep sessions separate when evaluating. If you randomly split adjacent frames from the same recording into train and test, the model can appear extremely accurate because nearly identical frames occur in both sets. That is data leakage.

A good custom gesture flow is: create label → record examples → validate quality → extract features → train/fit model → evaluate on a separate recording → save model/configuration → map gesture to an action.

• Collect variation intentionally: distance, rotation, slight finger variation, different backgrounds, and different sessions.

• Include negative examples: normal hand movements that should NOT trigger the custom gesture.

• Keep a reject/unknown state. A recognizer that always chooses one known class will eventually trigger incorrectly.

• Store the feature-extraction version with the model. Changing normalization later can invalidate an old model.

```
sample = {
"label": "my_gesture",
"features": feature_vector.tolist(),
"hand": "Right",
"session": "2026-09-29-session-01"
}
```

10. Dynamic gestures and time

A static classifier sees one frame. A dynamic gesture is a sequence X_1, X_2, …, X_t. Examples include swipes, waves, circles, and movements between two poses.

Maintain a rolling buffer of recent feature vectors. A sequence classifier can use the whole window, or simpler features can be computed from it.

Velocity: $v_t = (P_t - P_{t-1})/\Delta t$. Acceleration: $a_t = (v_t - v_{t-1})/\Delta t$. For real-time cameras, use timestamps rather than assuming every frame arrives at exactly the same interval.

Trajectory: track a stable point such as palm center. A swipe can be summarized by displacement, duration, direction, and peak speed.

Dynamic Time Warping (DTW): useful when two people perform the same motion at different speeds. DTW aligns sequences nonlinearly in time and calculates a distance between them. It is especially useful for template-style personalized dynamic gestures.

• Static gesture: classify the current normalized pose.

• Dynamic gesture: classify a sequence or trajectory.

• Hybrid gesture: require a start pose, movement, and end pose.

• Do not make the camera action fire on the first frame of a motion.

```
history.append(feature_vector)
if len(history) >= WINDOW:
sequence = np.asarray(history)
# Example sequence features:
delta = sequence[-1] - sequence[0]
speed = np.linalg.norm(np.diff(sequence, axis=0), axis=1)
peak_speed = speed.max()
```

11. Temporal stability and trigger logic

Recognition and triggering are separate problems. A classifier can be correct while still being too noisy for a camera shutter.

A robust trigger layer can require a candidate gesture to persist for N frames or a short duration, then fire once, enter cooldown, and wait for the gesture to disappear before allowing another trigger.

Majority voting: among the last N labels, choose the most common. Exponential smoothing: smooth continuous confidence scores. Hysteresis: use different thresholds for entering and leaving a state. Cooldown: prevent repeated actions immediately after a trigger.

Consider the latency budget: camera capture + hand detection + feature processing + classification + stability window + action execution. A stable system that takes too long will feel unresponsive.

```
state = "NONE"
consecutive = 0
cooldown_until = 0.0

# Pseudocode:
# if confidence >= enter_threshold:
# consecutive += 1
# else:
# consecutive = 0
# if consecutive >= required_frames and now >= cooldown_until:
# trigger()
# cooldown_until = now + cooldown_seconds
```

Layer	Question it answers
Classifier	What gesture does this frame/sequence resemble?
Confidence filter	Is the resemblance strong enough?
Stability filter	Has it remained stable long enough?
State machine	Is this a new gesture or the same one?
Cooldown	Should an action be temporarily blocked?

Layer	Question it answers
Action mapper	What camera action is assigned to this gesture?

12. Handedness, depth and difficult cases

Handedness should be treated as part of your data pipeline, not as an afterthought. If the product should accept either hand, define a canonical coordinate convention and test left/right recordings separately.

Depth is useful but noisy. Use z as one feature family rather than assuming it is perfectly metric. Compare image landmarks, relative 3D landmarks, and world landmarks experimentally.

Occlusion is a major failure mode. Fingers can overlap or disappear partially. Your classifier should have a confidence/rejection mechanism instead of confidently guessing from incomplete geometry.

Multiple hands require an explicit policy: which hand controls the app, whether gestures are allowed with either hand, and what happens when two hands are visible. Do not let the first detected hand silently change between frames.

• Track a hand identity over time if multiple hands are possible.

• Reject frames with insufficient/low-quality landmarks.

• Do not assume a missing landmark means the finger is closed.

• Test front-facing and rear-facing camera configurations separately.

• Test mirrored preview versus actual image processing; the UI preview may be mirrored while the processed frame is not.

13. Dataset design and evaluation

The quality of your dataset usually matters more than tiny differences between classifiers.

Labels: each example must have a precise meaning. Define exactly what counts as the gesture and what does not.

Positive examples: multiple people or sessions if the app is meant for general users; multiple sessions are especially important even for a single user.

Negative examples: natural hand movements, near-miss poses, transitions, partially visible hands, and no-hand frames.

Splits: train, validation, and test. Keep sessions or users separated when possible.

Metrics: accuracy is useful but insufficient. Track precision, recall, F1, confusion matrix, false triggers per minute, missed triggers, and trigger latency.

Open-set rejection: the system must be allowed to say “none of my known gestures” instead of forcing an answer.

Metric	Meaning	Why it matters for a camera app
Precision	Of triggers predicted, how many were correct?	Low precision means accidental photos/actions.
Recall	Of real gestures, how many were detected?	Low recall means missed commands.
F1	Balance of precision and recall	Useful for comparing classifiers.
False triggers/min	Wrong actions per minute	Directly reflects user annoyance.

Metric	Meaning	Why it matters for a camera app
Latency	Time from gesture completion to action	Reflects responsiveness.
Confusion matrix	Which gestures get mixed up?	Shows exactly what to improve.

14. Software architecture for File 3

Use small modules. This lets you replace a rule-based classifier with KNN/SVM/MLP later without rewriting normalization or the camera code.

• Landmark validation: check count, finite values, and expected ranges.

• Normalization: pure functions where possible. They should be easy to unit-test.

• Features: return a fixed-length vector with documented ordering.

• Classifier: only predicts; it should not trigger a camera.

• Temporal layer: handles history and motion.

• Action mapper: lives outside File 3 or in a higher-level controller.

• Configuration: thresholds and model paths should be external, especially because your UI will eventually let users customize them.

```
gesture_processor/
■■■ __init__.py
■■■ landmarks.py # indices, finger definitions, validation
■■■ normalization.py # translation/scale/rotation/handedness
■■■ geometry.py # vectors, distances, angles
■■■ features.py # feature vector construction
■■■ rules.py # interpretable baseline recognizer
■■■ classifier.py # ML model wrapper
■■■ temporal.py # rolling windows / DTW / motion features
■■■ smoothing.py # confidence smoothing and majority voting
■■■ state_machine.py # candidate/confirmed/cooldown logic
■■■ result.py # GestureResult data structure
■■■ config.py # thresholds and model settings
```

15. Implementation blueprint

Start with a deterministic baseline. Do not train a neural network before you can inspect your features and understand your errors.

Step 1: convert MediaPipe output into a clean NumPy array of shape (21,3).

Step 2: validate landmarks and reject unusable frames.

Step 3: center on the wrist or another chosen anchor.

Step 4: normalize scale.

Step 5: optionally canonicalize rotation and handedness.

Step 6: compute angles, distances, vectors, and ratios.

Step 7: build a documented feature vector.

Step 8: implement 4–6 rule-based gestures.

Step 9: log features and labels into a dataset.

Step 10: train a simple classifier such as KNN/SVM/Random Forest.

Step 11: add temporal smoothing and rejection.

Step 12: add a custom gesture recorder.

Step 13: add dynamic gesture support only after static gestures are reliable.

```
from dataclasses import dataclass

@dataclass
class GestureResult:
name: str
confidence: float
hand: str | None
stable: bool
timestamp: float

# The camera layer can consume this object without knowing how
# the gesture was recognized.
```

16. Experiments and learning roadmap

Do these experiments in order. Each experiment should produce a measurable result and teach you one concept.

Experiment 1 — Landmark viewer: draw all 21 points and indices. Verify your coordinate assumptions.

Experiment 2 — Translation test: move the same hand around the frame and confirm normalized coordinates remain nearly constant.

Experiment 3 — Scale test: move the hand toward and away from the camera. Compare raw and normalized features.

Experiment 4 — Rotation test: rotate the wrist and compare coordinate, angle, and distance representations.

Experiment 5 — Feature plots: record several gestures and inspect which features separate them.

Experiment 6 — Rule baseline: implement open palm, fist, pointing, peace, and pinch.

Experiment 7 — KNN/SVM/Random Forest comparison: use identical train/test splits and compare F1 and false-trigger rate.

Experiment 8 — Unknown rejection: feed unrelated poses and measure false positives.

Experiment 9 — Temporal stability: vary frame-count requirements and cooldown and measure latency versus false triggers.

Experiment 10 — Dynamic gesture: implement swipe using a rolling window; then investigate DTW.

Experiment 11 — Personalized gesture training: build the custom gesture recorder and test whether a model trained on one session works on another.

Experiment 12 — Real-world stress test: different distances, rotations, backgrounds, lighting, clothing, partial occlusion, and normal hand movements.

• Keep a CSV/JSON log of predictions and ground-truth labels.

• Save failed examples. Your failure set becomes the most valuable dataset for improving the system.

• Change one major variable at a time during experiments.

• Write down the feature version and classifier settings for every model you evaluate.

17. Debugging checklist

When the recognizer behaves badly, debug from the bottom upward.

Symptom	Likely cause	Check
Gesture changes when hand moves	No translation normalization	Plot wrist-centered landmarks
Gesture changes when closer	No/poor scale normalization	Inspect size ratios
Rotation breaks recognition	Axis-specific rules	Use angles/vectors/canonical frame
Left and right differ	Handedness handling	Mirror into canonical coordinates
Random labels	Landmark noise / classifier overconfidence	Plot confidence and reject low-confidence frames
Repeated photos	No state/cooldown	Implement confirmed → triggered → cooldown
Missed gestures	Threshold too strict / poor features	Inspect false negatives and feature distributions
High test score, bad real-world performance	Data leakage	Split by session/user
Two hands cause flicker	No hand identity policy	Track/select a hand explicitly
Motion gestures fail	Single-frame classifier	Use temporal window/trajectory/DTW

18. Further study and references

MediaPipe Hand Landmarker / Gesture Recognizer: study the official documentation for landmark outputs, handedness, world landmarks, running modes, and gesture recognition APIs. Google AI Edge documentation is the primary reference for current MediaPipe Tasks behavior.

MediaPipe Gesture Recognizer customization: useful for understanding how landmark-based gesture datasets and custom gesture models are structured.

Linear algebra: learn vectors, matrices, dot products, cross products, rotations, coordinate frames, and eigenvectors/PCA. These concepts directly explain normalization and feature construction.

Machine learning: learn train/validation/test splitting, standardization, overfitting, class imbalance, cross-validation, confusion matrices, precision/recall/F1, and probability calibration.

Time-series recognition: study rolling windows, velocity/acceleration, sequence classification, Dynamic Time Warping, and eventually HMMs or recurrent/transformer models if your project grows.

Computer vision: understand perspective, camera geometry, image coordinates, mirrored previews, and the effect of viewpoint and occlusion.

• Google AI Edge — MediaPipe Hand Landmarker documentation: https://ai.google.dev/edge/mediapipe/solutions/vision/hand_landmarker

• Google AI Edge — MediaPipe Gesture Recognizer: https://ai.google.dev/edge/mediapipe/solutions/vision/gesture_recognizer

• Google AI Edge — Gesture Recognizer customization: https://ai.google.dev/edge/mediapipe/solutions/customization/gesture_recognizer

• MediaPipe repository: https://github.com/google-ai-edge/mediapipe

• Scikit-learn documentation for KNN, SVM, Random Forest, preprocessing, and metrics: https://scikit-learn.org/stable/

Final cheat sheet: what File 3 should ultimately do

```
MediaPipe output
↓
validate landmarks
↓
canonicalize representation
• translation
• scale
• optional rotation
• optional handedness
↓
extract features
• coordinates
• vectors
• angles
• distances
• ratios
• motion
↓
classify
• rules / KNN / SVM / RF / MLP
↓
confidence + unknown rejection
↓
temporal stabilization
• smoothing
• majority vote
• state machine
• cooldown
↓
GestureResult
↓
action mapping
• photo
• start/stop video
• custom command
```

> The key mental model: do not ask “what are the x/y coordinates of this gesture?” Ask “what geometric relationships define this gesture, and which of those relationships should remain unchanged when the hand moves, scales, rotates, mirrors, or changes speed?”

Recommended learning order: landmarks → vectors → distances/angles → normalization → features → rules → dataset → KNN/SVM/RF → evaluation → temporal filtering → DTW → custom gesture recorder → production hardening.